

循序渐进，学习开发 xv6-riscv

第 13 章 RVOS2

汪辰



目录

01

控制台、终端和 Shell

02

xv6 控制台总体设计

03

控制台和串口驱动的具体实现

04

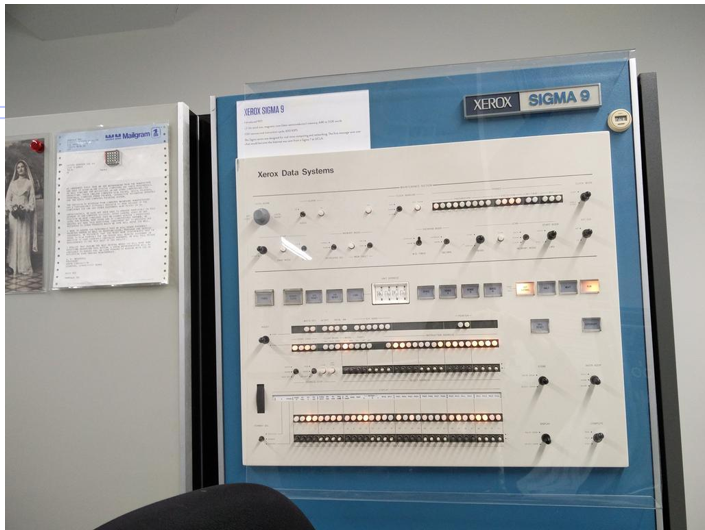
一个简单的 Shell



01

控制台、终端和 Shell

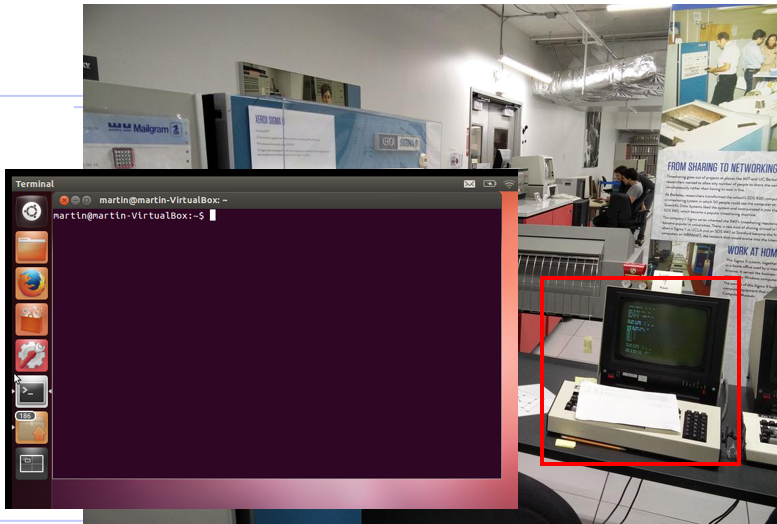
控制台 (Console)



终端 (Terminal)

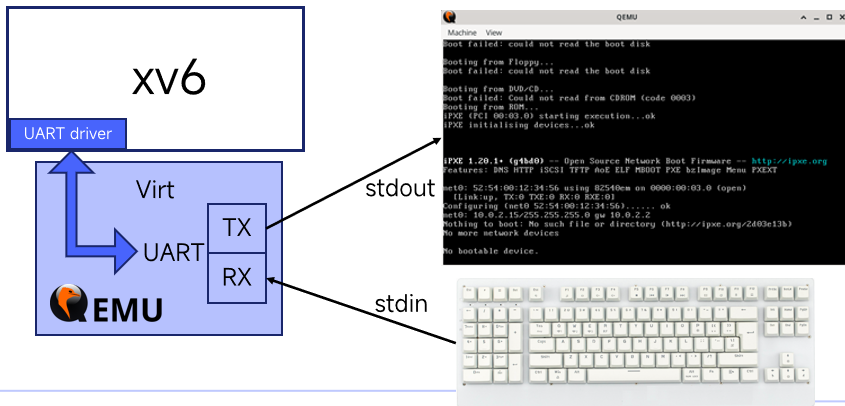


终端 (Terminal)



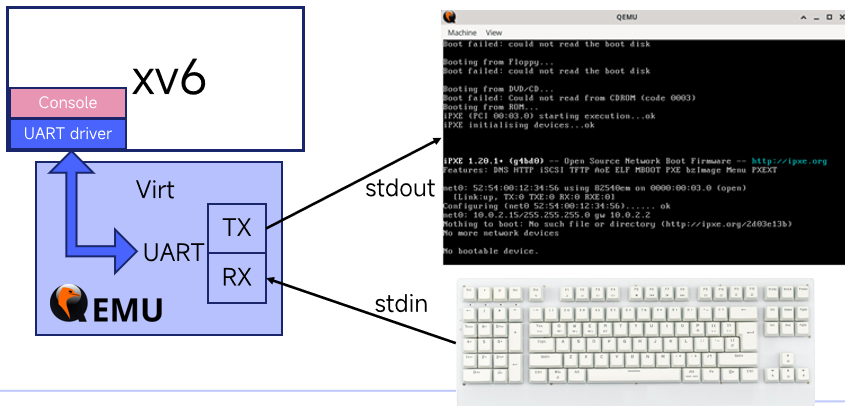
QEMU 的输入和输出

```
QEMUOPTS = -machine virt -bios none -kernel $K/kernel -m 128M -smp $(CPUS) -nographic
```



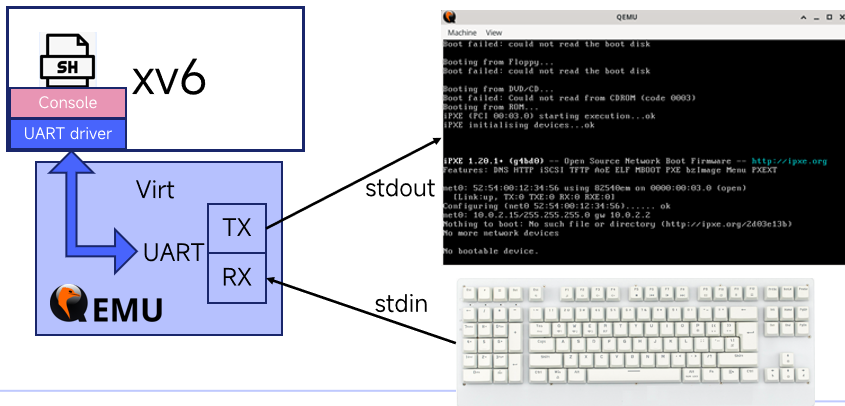
QEMU 的输入和输出

```
QEMUOPTS = -machine virt -bios none -kernel $K/kernel -m 128M -smp $(CPUS) -nographic
```



QEMU 的输入和输出

```
QEMUOPTS = -machine virt -bios none -kernel $K/kernel -m 128M -smp $(CPUS) -nographic
```

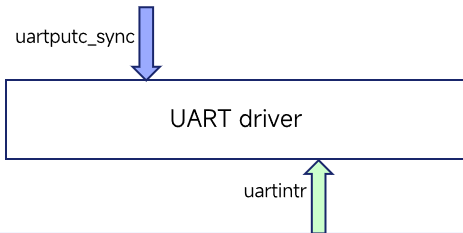




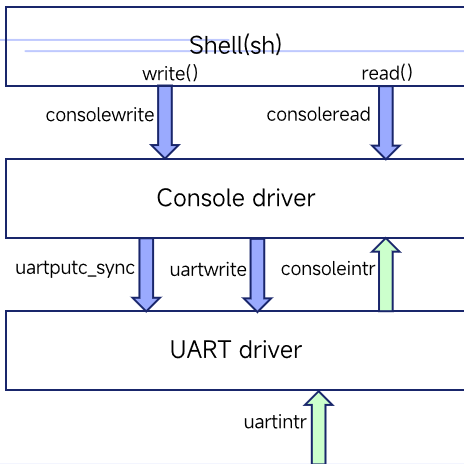
02

xv6 中控制台总体设计

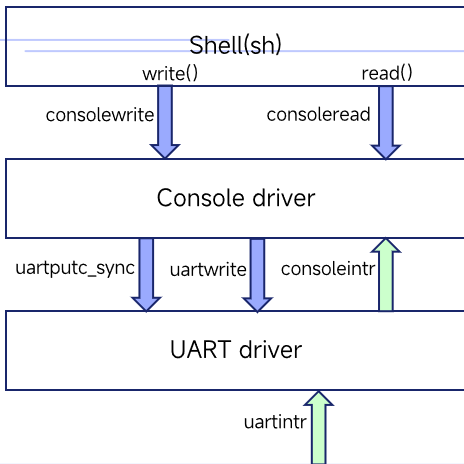
xv6 中的控制台总体设计



xv6 中的控制台总体设计



xv6 中的控制台总体设计



内部实现了一个行缓存

- 对普通的字符输入进行回显
- 分析并处理行缓存中的一些特殊的和行编辑相关的输入控制字符，例如，Backspace / Ctrl+H / Ctrl+U
- 当用户按下了 "Enter" 键，这体现为接受到一个 '\n' 字符时表示行编辑结束，此时控制台需要将行缓存中累积的数据交给上层应用处理。
- 其他特殊的控制字符处理，譬如 Ctrl+P



03

控制台和串口驱动的 具体实现

控制台结构体

```
struct {  
    struct spinlock lock;  
  
    // input  
#define INPUT_BUF_SIZE 128  
    char buf[INPUT_BUF_SIZE];  
    uint r; // Read index  
    uint w; // Write index  
    uint e; // Edit index  
} cons;
```

控制台结构体

```
struct {  
    struct spinlock lock;  
  
    // input  
#define INPUT_BUF_SIZE 128  
    char buf[INPUT_BUF_SIZE];  
    uint r; // Read index  
    uint w; // Write index  
    uint e; // Edit index  
} cons;
```

保证同时只有一个任务或者中断可以访问行缓存。

控制台结构体

```
struct {  
    struct spinlock lock;  
  
    // input  
#define INPUT_BUF_SIZE 128  
    char buf[INPUT_BUF_SIZE];  
    uint r; // Read index  
    uint w; // Write index  
    uint e; // Edit index  
} cons;
```

保证同时只有一个任务或者中断可以访问行缓存。

环形数组，用于存放输入的字符缓存。

控制台结构体

```
struct {  
    struct spinlock lock;  
  
    // input  
#define INPUT_BUF_SIZE 128  
    char buf[INPUT_BUF_SIZE];  
    uint r; // Read index  
    uint w; // Write index  
    uint e; // Edit index  
} cons;
```

保证同时只有一个任务或者中断可以访问行缓存。

环形数组，用于存放输入的字符缓存。

标记着已消费（read）的下边界，每消费一个（复制到 Shell）就 $r++$ ， r 和 w 之间是剩余的可以消费的数据。当 $r == w$ 时说明没有可读的数据。

控制台结构体

```
struct {  
    struct spinlock lock;  
  
    // input  
#define INPUT_BUF_SIZE 128  
    char buf[INPUT_BUF_SIZE];  
    uint r; // Read index  
    uint w; // Write index  
    uint e; // Edit index  
} cons;
```

保证同时只有一个任务或者中断可以访问行缓存。

环形数组，用于存放输入的字符缓存。

标记着已消费（read）的下边界，每消费一个（复制到 Shell）就 $r++$ ， r 和 w 之间是剩余的可以消费的数据。当 $r == w$ 时说明没有可读的数据。

标记着可消费的上边界，当 $r != w$ 时，意味着上层应用（Shell）可以消费了。

控制台结构体

```
struct {  
    struct spinlock lock;  
  
    // input  
#define INPUT_BUF_SIZE 128  
    char buf[INPUT_BUF_SIZE];  
    uint r; // Read index  
    uint w; // Write index  
    uint e; // Edit index  
} cons;
```

保证同时只有一个任务或者中断可以访问行缓存。

环形数组，用于存放输入的字符缓存。

标记着已消费（read）的下边界，每消费一个（复制到 Shell）就 $r++$ ， r 和 w 之间是剩余的可以消费的数据。当 $r == w$ 时说明没有可读的数据。

标记着可消费的上边界，当 $r != w$ 时，意味着上层应用（Shell）可以消费了。

标记着缓存的上边界，当消费条件满足时（譬如接收到一个 Enter）将 e 赋值给 w 使得 $r != w$ ，触发上层消费。

接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // .....
}
```

接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // .....
}
```

```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        int c = uartgetc();
        if(c == -1)
            break;
        consoleintr(c);
    }
}
```

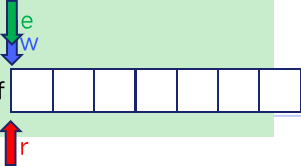
接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // .....
}
```

```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
            }
            break;
        }
    }
    // .....
}
```

接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
}
```



```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
        }
    }
    // .....
}
```

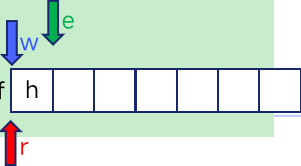

接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
}
```

```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
        }
    }
    // .....
}
```

接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
}
```

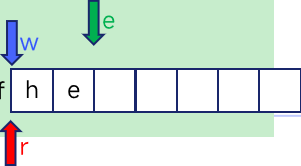


```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
            // .....
        }
    }
}
```



接收(read)方向

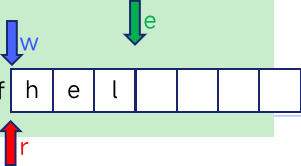
```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
}
```



```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
        }
    }
    // .....
}
```

接收(read)方向

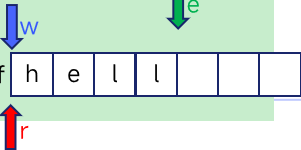
```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
}
```



```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
        }
    }
    // .....
}
```

接收(read)方向

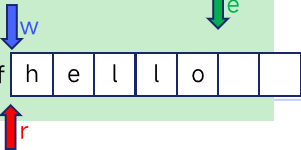
```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
    h e l l
```



```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
            // .....
        }
    }
}
```

接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
}
```



```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
        }
    }
    // .....
}
```

接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
}
```

```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
        }
    }
    // .....
}
```

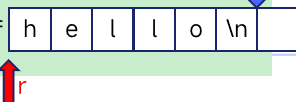
接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
    h e l l o \n
```

```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
            // .....
        }
    }
}
```


接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
}
```

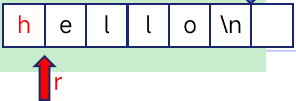


```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
            // .....
        }
    }
}
```

接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf

```



```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
            // .....
        }
    }
}
```

接收(read)方向

```
int consoleread(uint64 dst, int n)
{
    int c;
    char cbuf;
    // .....
    while(n > 0){
        // .....
        while(cons.r == cons.w);
        // .....
        c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
        // .....
        // copy the input byte to the user-space buffer.
        cbuf = c;
        *((char *)dst) = cbuf;
        dst++;
        --n;
        if(c == '\n'){
            break;
        }
    }
    // ..... cons.buf
```

h e l l o \n T

```
void uartintr(void)
{
    // .....
    // read and process incoming characters.
    while(1){
        void consoleintr(int c)
        {
            // .....
            switch(c){
                // .....
            }
            default:
                if(c != 0 && cons.e-cons.r < INPUT_BUF_SIZE){
                    c = (c == '\r') ? '\n' : c;
                    // echo back to the user.
                    consputc(c);
                    // store for consumption by consoleread().
                    cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
                    if(c == '\n' || c == C('D') || cons.e-cons.r == INPUT_BUF_SIZE){
                        // wake up consoleread() if a whole line (or end-of-file)
                        // has arrived.
                        cons.w = cons.e;
                    }
                }
                break;
            }
            // .....
        }
    }
}
```

发送(write)方向

```
int consolewrite(uint64 src, int n)
{
    char buf[32];
    int i = 0;

    while(i < n){
        int nn = sizeof(buf);
        if(nn > n - i)
            nn = n - i;
        memmove(buf, (const void *)src+i, nn);
        uartwrite(buf, nn);
        i += nn;
    }

    return i;
}
```

```
void uartwrite(char buf[], int n)
{
    acquire(&tx_lock);
    int i = 0;
    while(i < n){
        release(&tx_lock);
        while(tx_busy != 0);
        acquire(&tx_lock);
        WriteReg(THR, buf[i]);
        i += 1;
        tx_busy = 1;
    }
    release(&tx_lock);
}
```

```
// for transmission.
static struct spinlock tx_lock;
static int tx_busy; // is the UART busy sending?

void uartinit(void)
{
    // .....
    // enable transmit and receive interrupts.
    WriteReg(IER, IER_TX_ENABLE | IER_RX_ENABLE);
    initlock(&tx_lock, "uart");
}
```

```
void uartintr(void)
{
    ReadReg(ISR); // acknowledge the interrupt
    acquire(&tx_lock);
    if(ReadReg(LSR) & LSR_TX_IDLE){
        tx_busy = 0;
    }
    release(&tx_lock);
    // .....
}
```



04

一个简单的 Shell

一个简单的 Shell

```
void user_shell(void)
{
    static char buf[100];

    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0){
        char *cmd = buf;
        while (*cmd == ' ' || *cmd == '\t')
            cmd++;
        if (*cmd == '\n') // is a blank command
            continue;
        cmd[strlen(cmd) - 1] = '\0';
        runcmd(cmd);
    }
}
```

```
void userinit(void)
{
    struct proc *p;

    // create task0 and task1 .....

    p = allocproc(user_shell);
    if (p == 0)
        panic("userinit: allocproc task0 failed");
    safestrcpy(p->name, "shell", 6);
    p->state = RUNNABLE;
    release(&p->lock);
}
```

xv6 kernel is booting

hart 1 starting

hart 2 starting

\$

1 run task0

2 run task1

3 run shell

\$ ls

Running command: 'ls', unsupported!

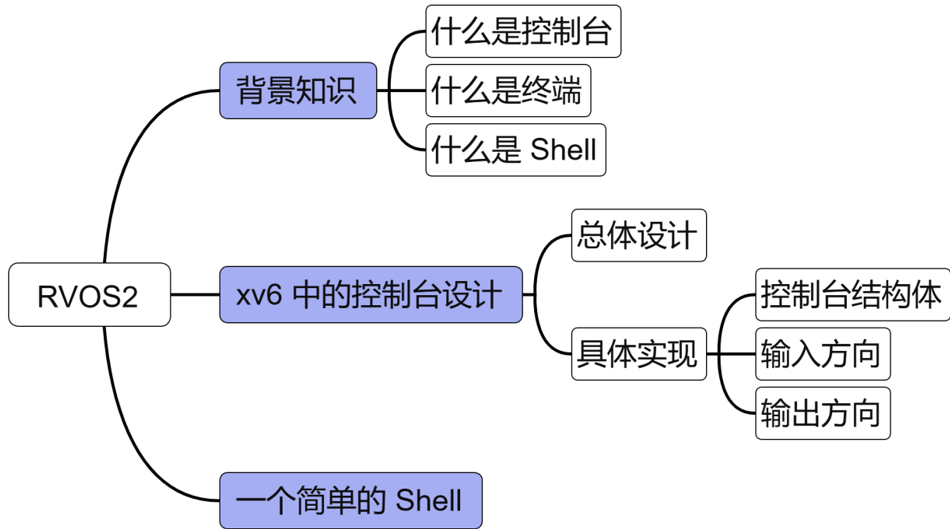
\$ hello

Running command: 'hello', unsupported!

\$



本章总结



谢谢

